

CUELO 하네스 벤치마크

같은 AI 모델에게 같은 코딩 과제를 주고, 그 AI를 부리는 "도구"만 바꿔 비교했다. CUELO(우리가 만든 도구), Claude Code, Codex 세 가지다. 정답 여부는 AI가 미리 볼 수 없는 채점 테스트로 판정했고, 모든 값은 여러 번 돌린 결과의 가운데 값이다.

한눈에 보기

42% 빠름

어려운 과제 · Claude Code 대비

같은 AI(Opus 5.5), 정답
같은 · 요금 6% 저렴

-37%

고친 뒤 초대형 과제
시간

25분 39초 → 16분 13초 ·
테스트 953 → 957/960

-43%

고친 뒤 보통 과제 요금

\$0.76 → \$0.44 · 정답

그대로

절반

하위 AI를 소형 모델로

요금 \$6.81 → \$3.37 · 대신

시간 2.6배

1. 같은 AI라도 도구가 결과를 바꾼다. 같은 Opus 5.5로 어려운 과제를 풀 때 CUELO가 가장 빨랐고, 정답률은 모든 과제에서 Claude Code와 같거나 높았다.
2. 측정하고, 고치고, 다시 측정했다. 1차 측정에서 찾은 병목을 고치자 분야별로 시간은 최대 37%, 사용량은 최대 43% 줄었다. 효과가 없던 시도 2개는 되돌렸다.
3. 하위 AI에게 맡기는 건 공짜가 아니다. 기능 하나 고치는 작은 일을 맡기면 요금이 4.4배가 됐다. 크고 급하지 않은 일은 소형 모델 하위 AI에 맡기면 요금이 절반으로 준다. 이때 하위 AI 몫은 총액의 4% 뿐이다.

아직 약한 곳: 수정 3개 묶음 과제는 사용량이 Claude Code보다 34% 많고, 보통 과제는 요금이 43% 비싸다.

공정성 메모: '혼자' 칸이지만 실제로 하위 AI를 쓴 두 경우(Claude Code 초대형 과제, Codex 어려운 과제)를 실행 기록으로 찾아 표에 표시했다. 같은 병렬 조건이면 초대형 과제 시간은 CUELO 16분 13초, Claude Code 16분 24초로 비슷하다.

읽는 순서: ① 한눈에 보기 → ② 과제별 한 줄 요약 → ③ 개선 전 vs 개선 후 → ④ 과제별 자세한 표 → ⑤ 혼자 할 때 vs 나눠 맡길 때. 초록 ▲는 CUELO에 유리, 빨강 ▼는 불리, 회색 =는 차이 5% 미만.

CUELO는 무엇인가

CUELO는 오픈소스 코딩 에이전트 코어 OMP(oh-my-pi) 위에 우리가 만든 하네스다. 하네스는 AI 모델에게 무엇을 먼저 읽히고, 어떤 도구를 주고, 언제 일을 나눠 맡길지 정하는 규칙·도구·운영 계층이다. 모델이 엔진이라면 하네스는 그 엔진을 모는 차체와 운전 규칙이다. 이 벤치마크는 같은 모델을 넣고 도구만 바꿔 비교했으므로, 결과 차이는 모델이 아니라 하네스에서 나온다.

층	하는 일
코어 · OMP	모델 호출, 파일 읽기·편집, 셀 실행, 하위 AI 호출 같은 기본 실행 엔진. 오픈소스를 버전 고정해 쓰고 필요한 곳만 고쳤다(예: 출력 한도 64K 제한 해제, 한도에 끊긴 생각 이어 가기).
하네스 · 우리가 만든 부분	작업 규칙(항상 신는 규칙은 짧게, 나머지는 필요할 때 읽기), 역할 둘(Main은 요구·검수·최종 판정, Maker는 구현·검증), 발주 라우터 JEV(작업 등급과 계정별 사용량 여유를 보고 하위 AI의 모델·추론 강도를 추천), 조기 종료·위험 작업 가드.
화면 · CUELO 웹	브라우저에서 쓰는 작업 화면. 계정별 사용량 미터, 하위 AI 진행 상황, 옆 채널 질문을 한곳에서 본다.

Claude Code·Codex와 무엇이 다른가

	Claude Code	Codex	CUELO
메인 AI 모델	Anthropic(Claude)	OpenAI(GPT)	여러 회사 모델 중 선택
하위 AI 모델	같은 회사 모델 안에서	같은 회사 모델 안에서	역할마다 다른 회사 모델을 섞음(예: 메인 Opus + 하위 AI GPT-6 Luna·DeepSeek)
누구에게 맡길지	메인이 판단	메인이 판단	JEV가 작업 등급·계정 사용량 여유로 모델·강도를 추천, 메인이 확정
규칙·검수	사용자 설정 파일	사용자 설정 파일	역할·승인 경계·검수 계약을 규칙으로 고정하고 가드가 검사

과제별 한 줄 요약

초록 ▲는 CUELO가 유리, 빨강 ▼는 불리, 회색 =는 비슷함(차이 5% 미만)이다. 비교 상대는 같은 AI를 쓴 Claude Code다.

- 어려운 과제 · 안전 장치 새로 만들고 실제로 연결하기 — 시간 ▲ 42% 더 빠름 · 사용량 = 비슷함 · 요금 ▲ 6% 더 싼 · 정답률은 같음
- 묶음 과제(2개) · 서로 다른 수정 두 개 한 번에 — 시간 ▲ 10% 더 빠름 · 사용량 ▼ 20% 더 싼 · 요금 ▼ 16% 더 비쌌 · 정답률은 같음
- 묶음 과제(3개) · 서로 다른 수정 세 개 한 번에 — 시간 = 비슷함 · 사용량 ▼ 35% 더 싼 · 요금 ▼ 26% 더 비쌌 · 정답률은 같음
- 보통 과제 · 기존 기능 하나 고치기 — 시간 ▼ 6% 더 느림 · 사용량 ▼ 38% 더 싼 · 요금 ▼ 43% 더 비쌌 · 정답률은 같음
- 대규모 과제 · 스프레드시트 계산 엔진 새로 만들기 — 시간 ▲ 10% 더 빠름 · 사용량 = 비슷함 · 요금 ▲ 8% 더 싼 · 맞힌 테스트 비율 CUELO 98.3% · Claude Code 97.5%
- 초대형 과제 · 서로 독립인 패키지 6개 새로 만들기 — 시간 ▼ 53% 더 느림 · 사용량 ▲ 24% 덜 싼 · 요금 ▲ 19% 더 싼 · 정답률은 같음 (주의: Claude Code는 하위 AI 6개 병렬, CUELO는 발주 실패로 혼자 직렬이었다. 같은 병렬 조건이면 CUELO 16분 13초 대 Claude Code 16분 24초로 비슷함)

개선 전 vs 개선 후 (분야별)

1차 측정에서 찾은 병목을 고친 뒤 같은 과제·같은 모델로 다시 잴다. 개선 전 기록은 지우지 않고 그대로 둔다. 회색은 개선 전, 주황은 개선 후, 연보라는 같은 조건의 Claude Code(참고)다. 모든 값은 반복 실행의 가운데 값이다.

걸린 시간 변화 (개선 전 대비)

어려운 과제		-25%
보통 과제 (Opus)		+4%
보통 과제 (Luna)		-5%
초대형 과제 (위임)		-37%

AI 사용량 변화 (개선 전 대비)

어려운 과제		-11%
보통 과제 (Opus)		-43%
보통 과제 (Luna)		-24%
초대형 과제 (위임)		-20%

API 환산 요금 변화 (개선 전 대비)

어려운 과제		-11%
보통 과제 (Opus)		-43%
보통 과제 (Luna)		-25%
초대형 과제 (위임)		-16%

초록 막대는 줄어듦(좋음), 빨강 막대는 늘어남(나쁨)이다.

어려운 과제 · 안전 장치 새로 만들고 실제로 연결하기

CUELO · Opus 5.5 high, 각 3회 · 배치 oe-hard-3way → oe-fair

고친 것: 측정 조건 결함 수정: 과제 폴더의 규칙 파일(약 3.5만 자)이 CUELO에만 자동으로 읽혀 요청마다 약 2.8만 토큰이 더 들었다. 모든 도구에서 숨겨 조건을 맞췄다.

시간 ▲ 25% 단축 · AI 사용량 ▲ 11% 줄어듦 · API 환산 요금 ▲ 11% 줄어듦 · 정답 3/3 → 3/3

걸린 시간 — 짧을수록 좋음

개선 전		8분 50초
개선 후		6분 35초
Claude Code (참고)		11분 19초

AI 사용량 — 짧을수록 좋음

개선 전		808K
개선 후		720K
Claude Code (참고)		745K

API 환산 요금 — 짧을수록 좋음

개선 전		\$2.37
개선 후		\$2.11
Claude Code (참고)		\$2.26

보통 과제 · 기존 기능 하나 고치기 (Opus)

CUELO · Opus 5.5 medium, 각 3회 · 배치 oe-lever4 → oe-lever-1830

고친 것: 요청마다 실행 규칙·도구 설명 축소와 코어 18.3.0(도구 정의 축소)을 함께 적용했다. 두 효과는 분리되지 않는다.

시간 = 비슷함 · AI 사용량 ▲ 43% 줄어듦 · API 환산 요금 ▲ 43% 줄어듦 · 정답 3/3 → 3/3

걸린 시간 — 짧을수록 좋음

개선 전		47초
개선 후		49초
Claude Code (참고)		48초

AI 사용량 — 짧을수록 좋음

개선 전		188K
개선 후		107K
Claude Code (참고)		86K

API 환산 요금 — 짧을수록 좋음

개선 전		\$0.76
개선 후		\$0.44
Claude Code (참고)		\$0.32

보통 과제 · 기존 기능 하나 고치기 (소형 모델)

CUELO · GPT-6 Luna medium, 각 3회 · 배치 oe-models → oe-lever4

고친 것: 독립 읽기 묶음 · 거절된 편집 재전송 규칙: 파일을 하나씩 따로 읽고, 편집이 거절되면 다시 읽느라 요청 수가 늘던 문제를 고쳤다.

시간 ▲ 5% 단축 · AI 사용량 ▲ 24% 줄어듦 · API 환산 요금 ▲ 25% 줄어듦 · 정답 3/3 → 3/3

걸린 시간 — 짧을수록 좋음		AI 사용량 — 짧을수록 좋음		API 환산 요금 — 짧을수록 좋음	
개선 전		1분 19초	개선 전		181K 개선 전 \$0.0175
개선 후		1분 15초	개선 후		137K 개선 후 \$0.0131

초대형 과제 · 패키지 6개를 하위 AI 6명에게 병렬 위임

CUELO · 메인 Opus 5.5 high + Opus Maker 6, 각 1회 · 배치 oe-mega-redo → oe-mega-128k

고친 것: 출력 한도 64K 제한을 공식 한도 128K로 풀고, 한도에 걸려 끊긴 생각을 기록 요약이 지우지 않고 이어 가게 했다(개선 전 한 Maker가 613초 + 421초를 잃었다).

시간 ▲ 37% 단축 · AI 사용량 ▲ 20% 줄어듦 · API 환산 요금 ▲ 16% 줄어듦 · 숨김 테스트 953/960 → 957/960

걸린 시간 — 짧을수록 좋음		AI 사용량 — 짧을수록 좋음		API 환산 요금 — 짧을수록 좋음	
개선 전		25분 39초	개선 전		5.08M 개선 전 \$16.01
개선 후		16분 13초	개선 후		4.07M 개선 후 \$13.47
Claude Code(병렬)		16분 24초	Claude Code(병렬)		3.21M Claude Code(병렬) \$8.73

모든 과제 공통 · 매 요청에 실는 고정 분량 (Opus 5.5 첫 요청)

고친 것: 문서 카탈로그·위임 규칙·브라우저 도구 설명을 필요할 때만 읽게 하고 규칙을 영어 사본으로 바꾼 누적 효과다. 요청마다 다시 실리는 양이라 모든 과제의 사용량에 곱해진다.

첫 요청 ▲ 41% 줄어듦 · Claude Code와의 차이 26,423 → 7,214 토큰

첫 요청 크기(토큰) — 짧을수록 좋음	
개선 전 (9/24)	46,705
개선 후 (9/25)	27,496
Claude Code (참고)	20,282

시도했지만 되돌린 것 · 묶음 과제(3개)

CUELO · Opus 5.5 high, 각 3회 · 배치 oe-night-base / oe-night-after / oe-narrow-read

검색·읽기 결과를 작게 가져오면 요청당 문맥은 13~14% 줄었지만, AI가 도구를 더 자주 불러
총사용량이 grep 문맥 0줄 ▼ 8% 늘어남, 좁게 읽기 = 비슷함 었다. 효과가 없어 둘 다 되돌렸다.
정답은 모두 3/3.

걸린 시간 — 짧을수록 좋음		AI 사용량 — 짧을수록 좋음		API 환산 요금 — 짧을수록 좋음	
기준	5분 55초	기준	958K	기준	\$2.95
grep 문맥 0줄	7분 7초	grep 문맥 0줄	1.03M	grep 문맥 0줄	\$3.07
좁게 읽기 지침	7분 18초	좁게 읽기 지침	996K	좁게 읽기 지침	\$2.98

어려운 과제 · 안전 장치 새로 만들고 실제로 연결하기

한 줄 요약: 같은 AI(Opus 5.5)를 쓸 때 CUELO는 Claude Code보다 시간 ▲ 42% 더 빠름, AI 사용량 = 비슷함, API 환산 요금 ▲ 6% 더 싼, 정답률은 같음.

조건 차이 주의: 이 칸의 Codex는 3회 중 2회 하위 AI에게 구현을 맡겼다(실행 기록의 하위 AI 대기 호출 15회·8회와 "구현을 위임" 메시지, 나머지 1회는 혼자). 이 Codex 결과는 측정 조건 수정 전에 낸 것이라 과제 폴더의 우리 규칙 파일(위임 규칙 포함)이 Codex에도 읽혔고, 그 규칙을 따른 것이다. CUELO는 조건 수정 뒤 혼자 한 결과다. Codex의 하위 AI 사용량은 기록에 따로 남지 않아 사용량·요금이 실제보다 적게 잡혔을 수 있다.

순위	도구 · AI 모델	정답 횟수	걸린 시간 (Claude Code 대비)	AI 사용량 (Claude Code 대비)	API 환산 요금 (Claude Code 대비)
1	● CUELO · Opus 5.5 high	3/3 전부 정답	6분 35초 ▲ 42% 빠름	720K = 비슷함	\$2.11 ▲ 6% 싼
2	● Claude Code · Opus 5.5 high	3/3 전부 정답	11분 19초	745K	\$2.26
3	● Codex · Daybreak Blue high (3회 중 2회 위임)	3/3 전부 정답	21분 40초 ▼ 91% 느림	961K ▼ 29% 많음	\$3.65 ▼ 62% 비쌌

정답률 — 길수록 좋음		걸린 시간 — 짧을수록 좋음		AI 사용량 — 짧을수록 좋음	
CUELO · Opus 5.5 high	100%	CUELO · Opus 5.5 high	6분 35초	CUELO · Opus 5.5 high	720K
Claude Code · Opus ...	100%	Claude Code · Opus ...	11분 19초	Claude Code · Opus ...	745K
Codex · 2/3회 위임	100%	Codex · 2/3회 위임	21분 40초	Codex · 2/3회 위임	961K

묶음 과제(2개) · 서로 다른 수정 두 개 한 번에

한 줄 요약: 같은 AI(Opus 5.5)를 쓸 때 CUELO는 Claude Code보다 시간 ▲ 10% 더 빠름 , AI 사용량 ▼ 20% 더 썸 , API 환산 요금 ▼ 16% 더 비쌘 , 정답률은 같음.

순위	도구 · AI 모델	정답 횟수	걸린 시간 (Claude Code 대비)	AI 사용량 (Claude Code 대비)	API 환산 요금 (Claude Code 대비)
1	● CUELO · Opus 5.5 high	3/3 전부 정답	1분 53초 ▲ 10% 빠름	281K ▼ 20% 많음	\$0.98 ▼ 16% 비쌘
2	● Claude Code · Opus 5.5 high	3/3 전부 정답	2분 5초	234K	\$0.84
3	● Codex · Daybreak Blue high	3/3 전부 정답	4분 4초 ▼ 95% 느림	240K = 비슷함	\$0.84 = 비슷함

정답률 — 길수록 좋음

CUELO · Opus 5.5 high		100%
Claude Code · Opus ...		100%
Codex · Daybreak Blu...		100%

걸린 시간 — 짧을수록 좋음

CUELO · Opus 5.5 high		1분 53초
Claude Code · Opus ...		2분 5초
Codex · Daybreak Blu...		4분 4초

AI 사용량 — 짧을수록 좋음

CUELO · Opus 5.5 high		281K
Claude Code · Opus ...		234K
Codex · Daybreak Blu...		240K

묶음 과제(3개) · 서로 다른 수정 세 개 한 번에

한 줄 요약: 같은 AI(Opus 5.5)를 쓸 때 CUELO는 Claude Code보다 시간 = 비슷함 , AI 사용량 ▼ 35% 더 싼 , API 환산 요금 ▼ 26% 더 비쌌 , 정답률은 같음.

순위	도구 · AI 모델	정답 횟수	걸린 시간 (Claude Code 대비)	AI 사용량 (Claude Code 대비)	API 환산 요금 (Claude Code 대비)
1	CUELO · Opus 5.5 high	3/3 전부 정답	6분 59초 = 비슷함	1.08M ▼ 35% 많음	\$3.12 ▼ 26% 비쌌
2	Claude Code · Opus 5.5 high	3/3 전부 정답	7분 11초	805K	\$2.48
3	Codex · Daybreak Blue high	2/3 일부 실패	7분 42초 ▼ 7% 느림	619K ▲ 23% 적음	\$2.25 ▲ 9% 싼

정답률 — 길수록 좋음

CUELO · Opus 5.5 high		100%
Claude Code · Opus ...		100%
Codex · Daybreak Blu...		67%

걸린 시간 — 짧을수록 좋음

CUELO · Opus 5.5 high		6분 59초
Claude Code · Opus ...		7분 11초
Codex · Daybreak Blu...		7분 42초

AI 사용량 — 짧을수록 좋음

CUELO · Opus 5.5 high		1.08M
Claude Code · Opus ...		805K
Codex · Daybreak Blu...		619K

보통 과제 · 기존 기능 하나 고치기

한 줄 요약: 같은 AI(Opus 5.5)를 쓸 때 CUELO는 Claude Code보다 시간 ▼ 6% 더 느림 , AI 사용량 ▼ 38% 더 씀 , API 환산 요금 ▼ 43% 더 비쌌 , 정답률은 같음.

순위	도구 · AI 모델	정답 횟수	걸린 시간 (Claude Code 대비)	AI 사용량 (Claude Code 대비)	API 환산 요금 (Claude Code 대비)
1	● Claude Code · Opus 5.5 medium	3/3 전부 정답	48초	86K	\$0.32
2	● CUELO · Opus 5.5 medium	3/3 전부 정답	51초 ▼ 6% 느림	119K ▼ 38% 많음	\$0.47 ▼ 43% 비쌌
3	● Codex · GPT-6 Luna medium	3/3 전부 정답	1분 2초 ▼ 29% 느림	56K ▲ 35% 적음	\$0.01 ▲ 98% 쌌

정답률 — 길수록 좋음

Claude Code · Opus ...		100%
CUELO · Opus 5.5 m...		100%
Codex · GPT-6 Luna ...		100%

걸린 시간 — 짧을수록 좋음

Claude Code · Opus ...		48초
CUELO · Opus 5.5 m...		51초
Codex · GPT-6 Luna ...		1분 2초

AI 사용량 — 짧을수록 좋음

Claude Code · Opus ...		86K
CUELO · Opus 5.5 m...		119K
Codex · GPT-6 Luna ...		56K

대규모 과제 · 스프레드시트 계산 엔진 새로 만들기

한 줄 요약: 같은 AI(Opus 5.5)를 쓸 때 CUELO는 Claude Code보다 시간 ▲ 10% 더 빠름 , AI 사용량 = 비슷함 , API 환산 요금 ▲ 8% 더 싼 , 맞힌 테스트 비율 CUELO 98.3% · Claude Code 97.5%.

순위	도구 · AI 모델	정답 횟수	걸린 시간 (Claude Code 대비)	AI 사용량 (Claude Code 대비)	API 환산 요금 (Claude Code 대비)
1	Codex · Daybreak Blue high	0/2 테스트 99.3% 맞힘	12분 31초 ▼ 28% 느림	462K ▲ 34% 적음	\$1.40 ▲ 33% 싼
2	CUELO · Opus 5.5 high	0/2 테스트 98.3% 맞힘	8분 47초 ▲ 10% 빠름	666K = 비슷함	\$1.93 ▲ 8% 싼
3	Claude Code · Opus 5.5 high	0/2 테스트 97.5% 맞힘	9분 48초	700K	\$2.10

맞힌 테스트 비율 — 길수록 좋음 (모두 만점은 못 받음)

Codex · Daybreak Blue high		99.3%
CUELO · Opus 5.5 high		98.3%
Claude Code · Opus 5.5 high		97.5%

걸린 시간 — 짧을수록 좋음

Codex · Daybreak Blue high		12분 31초
CUELO · Opus 5.5 high		8분 47초
Claude Code · Opus 5.5 high		9분 48초

AI 사용량 — 짧을수록 좋음

Codex · Daybreak Blue high		462K
CUELO · Opus 5.5 high		666K
Claude Code · Opus 5.5 high		700K

초대형 과제 · 서로 독립인 패키지 6개 새로 만들기

한 줄 요약: 같은 AI(Opus 5.5)를 쓸 때 CUELO는 Claude Code보다 시간 ▼ 53% 더 느림 , AI 사용량 ▲ 24% 덜 씀 , API 환산 요금 ▲ 19% 더 싼 , 정답률은 같음.

조건 차이 주의: 이 칸의 Claude Code는 혼자 한 것이 아니다. 스스로 하위 AI 6개를 병렬로 불러 패키지를 나눠 맡겼다(세션 기록의 subagent 6개, AI 누적 작업 시간 2,528초 대 실제 16분 24초). CUELO도 위임을 골랐지만 당시 설정 오류로 발주가 실패해 혼자 직렬로 처리했다. 그래서 이 표의 시간 차이는 도구 성능이 아니라 직렬 대 병렬의 차이다. 같은 병렬 조건인 CUELO 전부 위임(개선 후)은 16분 13초로 Claude Code 16분 24초와 비슷했다(요금은 \$13.47 대 \$8.73).

순위	도구 · AI 모델	정답 횟수	걸린 시간 (Claude Code 대비)	AI 사용량 (Claude Code 대비)	API 환산 요금 (Claude Code 대비)
1	● CUELO · Opus 5.5 high	0/1 테스트 99.8% 맞힘	25분 2초 ▼ 53% 느림	2.43M ▲ 24% 적음	\$7.10 ▲ 19% 싼
2	● Claude Code · Opus 5.5 high (하위 AI 6개 병렬)	0/1 테스트 99.5% 맞힘	16분 24초	3.21M	\$8.73
3	● Codex · Daybreak Blue high	0/1 테스트 98.5% 맞힘	25분 54초 ▼ 58% 느림	1.27M ▲ 60% 적음	\$4.31 ▲ 51% 싼

맞힌 테스트 비율 — 길수록 좋음 (모두 만점은 못 받음)		걸린 시간 — 짧을수록 좋음		AI 사용량 — 짧을수록 좋음	
CUELO · Opus 5.5 high	99.8%	CUELO · Opus 5.5 high	25분 2초	CUELO · Opus 5.5 high	2.43M
Claude Code(병렬)	99.5%	Claude Code(병렬)	16분 24초	Claude Code(병렬)	3.21M
Codex · Daybreak Blue high	98.5%	Codex · Daybreak Blue high	25분 54초	Codex · Daybreak Blue high	1.27M

메인 혼자 할 때 vs 전부 나눠 맡길 때

무엇을 비교했나: "메인 혼자"는 메인 AI가 하위 AI 없이 직접 구현한 경우다. 기록상 대부분 하위 AI를 쓰지 않았지만 예외가 둘 있다. 초대형 과제의 Claude Code는 스스로 하위 AI 6개를 병렬로 불렀고, 어려운 과제의 Codex는 3회 중 2회 하위 AI에게 구현을 맡겼다(당시 과제 폴더의 규칙 파일이 보인 조건). 초대형 과제의 CUELO는 위임을 시도했으나 당시 설정 오류로 하위 AI 후보를 받지 못해 혼자 직렬로 했다. "전부 위임"은 직접 고치지 말고 하위 AI(Maker·서브에이전트)에게 맡기라고 강제한 경우다. 비교 칩은 같은 도구의 메인 혼자 결과 대비다. 하위 AI 모델이 다르면 토큰 단가가 수십 배 다르므로 "AI 사용량"이 아니라 "API 환산 요금"으로 비교한다. Codex는 하위 AI의 사용량이 기록에 따로 나오지 않아 사용량·요금이 실제보다 적게 잡혔을 수 있다. Claude Code는 실행 기록에 중간 도구 호출이 남지 않아 실제 위임 여부를 확인하지 못했다. Codex 보통 과제 위임 2회는 하위 AI 생성 오류(도구 자체 문제)로 코드를 전혀 고치지 못해 표에서 뺐다.

묶음 과제(3개) · 서로 다른 수정 세 개 한 번에

도구 · 메인 모델	방식	정답	걸린 시간 (메인 혼자 대비)	AI 사용량 (메인 혼자 대비)	API 환산 요금 (메인 혼자 대비)
● Claude Code · Opus 5.5 high	메인 혼자	3/3 전부 정답	7분 11초	805K	\$2.48
● Codex · Daybreak Blue high	메인 혼자	2/3 일부 실패	7분 42초	619K	\$2.25
● CUELO · Opus 5.5 high	메인 혼자	3/3 전부 정답	6분 59초	1.08M	\$3.12
● CUELO · Opus 5.5 high	전부 위임 · Daybreak Blue Maker	1/2 일부 실패	20분 54초 ▼ 199% 느림	2.63M ▼ 142% 많음	\$8.30 ▼ 166% 비쌌
● CUELO · Opus 5.5 high	전부 위임 · DeepSeek 4.1 Flash Maker	2/2 전부 정답	27분 27초 ▼ 293% 느림	5.22M ▼ 381% 많음	\$5.29 ▼ 69% 비쌌
● CUELO · Opus 5.5 high	전부 위임 · GPT-6 Luna Maker	2/2 전부 정답	25분 4초 ▼ 259% 느림	2.76M ▼ 154% 많음	\$3.37 ▼ 8% 비쌌
● CUELO · Opus 5.5 high	전부 위임 · Opus 5.5 Maker	2/2 전부 정답	9분 40초 ▼ 38% 느림	2.07M ▼ 91% 많음	\$6.81 ▼ 118% 비쌌

어려운 과제 · 안전 장치 새로 만들고 실제로 연결하기

도구 · 메인 모델	방식	정답	걸린 시간 (메인 혼자 대비)	AI 사용량 (메인 혼자 대비)	API 환산 요금 (메인 혼자 대비)
● CUELO · Opus 5.5 high	메인 혼자	3/3 전부 정답	6분 35초	720K	\$2.11
● Claude Code · Opus 5.5 high	메인 혼자	3/3 전부 정답	11분 19초	745K	\$2.26
● Codex · Daybreak Blue high	메인 혼자라고 지시 · 3회 중 2회 위임	3/3 전부 정답	21분 40초	961K	\$3.65
● CUELO · Opus 5.5 high	전부 위임 · GPT-6 Luna Maker	0/1 테스트 99.5% 맞힘	29분 59초 ▼ 355% 느림	2.97M ▼ 312% 많음	\$2.90 ▼ 37% 비쌘
● Claude Code · Opus 5.5 high	전부 위임 · 자체 서버에 이전트	1/2 일부 실패	21분 55초 ▼ 94% 느림	1.88M ▼ 152% 많음	\$4.88 ▼ 116% 비쌘
● Codex · Daybreak Blue high	전부 위임 · 자체 서버에 이전트	2/2 전부 정답	20분 2초 ▲ 8% 빠름	341K ▲ 65% 적음	\$1.23 ▲ 66% 쌘
● CUELO · Opus 5.5 high	전부 위임 · Opus 5.5 Maker	2/2 전부 정답	11분 37초 ▼ 76% 느림	1.50M ▼ 108% 많음	\$4.47 ▼ 111% 비쌘

보통 과제 · 기존 기능 하나 고치기

도구 · 메인 모델	방식	정답	걸린 시간 (메인 혼자 대비)	AI 사용량 (메인 혼자 대비)	API 환산 요금 (메인 혼자 대비)
● CUELO · Opus 5.5 medium	메인 혼자	3/3 전부 정답	51초	119K	\$0.47
● Codex · GPT-6 Luna medium	메인 혼자	3/3 전부 정답	1분 2초	56K	\$0.01

도구 · 메인 모델	방식	정답	걸린 시간 (메인 혼자 대비)	AI 사용량 (메인 혼자 대비)	API 환산 요금 (메인 혼자 대비)
● Claude Code · Opus 5.5 medium	메인 혼자	3/3 전부 정답	48초	86K	\$0.32
● CUELO · Opus 5.5 medium	전부 위임 · GPT-6 Luna Maker	2/2 전부 정답	4분 29초 ▼ 426% 느림	488K ▼ 310% 많음	\$1.13 ▼ 142% 비쌌
● Claude Code · Opus 5.5 medium	전부 위임 · 자체 서버에 이전트	2/2 전부 정답	1분 37초 ▼ 101% 느림	147K ▼ 70% 많음	\$0.46 ▼ 43% 비쌌
● CUELO · Opus 5.5 medium	전부 위임 · Opus 5.5 Maker	2/2 전부 정답	3분 51초 ▼ 352% 느림	586K ▼ 393% 많음	\$2.05 ▼ 341% 비쌌

대규모 과제 · 스프레드시트 계산 엔진 새로 만들기

도구 · 메인 모델	방식	정답	걸린 시간 (메인 혼자 대비)	AI 사용량 (메인 혼자 대비)	API 환산 요금 (메인 혼자 대비)
● Claude Code · Opus 5.5 high	메인 혼자	0/2 테스트 97.5% 맞힘	9분 48초	700K	\$2.10
● Codex · Daybreak Blue high	메인 혼자	0/2 테스트 99.3% 맞힘	12분 31초	462K	\$1.40
● CUELO · Opus 5.5 high	메인 혼자	0/2 테스트 98.3% 맞힘	8분 47초	666K	\$1.93
● Claude Code · Opus 5.5 high	전부 위임 · 자체 서버에 이전트	0/2 테스트 98.3% 맞힘	12분 18초 ▼ 25% 느림	861K ▼ 23% 많음	\$2.45 ▼ 17% 비쌌
● Codex · Daybreak Blue high	전부 위임 · 자체 서버에 이전트	0/2 테스트 99.0% 맞힘	20분 29초 ▼ 64% 느림	369K ▲ 20% 적음	\$1.36 = 비슷함
● CUELO · Opus 5.5 high	전부 위임 · Opus 5.5 Maker	0/2 테스트 97.2% 맞힘	20분 11초 ▼ 130% 느림	2.86M ▼ 329% 많음	\$8.54 ▼ 344% 비쌌

초대형 과제 · 서로 독립인 패키지 6개 새로 만들기

도구 · 메인 모델	방식	정답	걸린 시간 (메인 혼자 대비)	AI 사용량 (메인 혼자 대비)	API 환산 요금 (메인 혼자 대비)
● Claude Code · Opus 5.5 high	자율 · 하위 AI 6개 병렬	0/1 테스트 99.5% 맞힘	16분 24초	3.21M	\$8.73
● Codex · Daybreak Blue high	메인 혼자	0/1 테스트 98.5% 맞힘	25분 54초	1.27M	\$4.31
● CUELO · Opus 5.5 high	메인 혼자	0/1 테스트 99.8% 맞힘	25분 2초	2.43M	\$7.10
● Claude Code · Opus 5.5 high	전부 위임 · 자체 서버에 이전트	0/1 테스트 99.4% 맞힘	23분 45초 ▼ 45% 느림	4.47M ▼ 39% 많음	\$12.20 ▼ 40% 비쌘
● Codex · Daybreak Blue high	전부 위임 · 자체 서버에 이전트	0/1 테스트 99.3% 맞힘	22분 57초 ▲ 11% 빠름	578K ▲ 55% 적음	\$2.14 ▲ 50% 쌘
● CUELO · Opus 5.5 high	전부 위임 · Opus 5.5 Maker	0/1 테스트 99.7% 맞힘	16분 13초 ▲ 35% 빠름	4.07M ▼ 67% 많음	\$13.47 ▼ 90% 비쌘
● CUELO · gpt-6-astra high	전부 위임 · Opus 5.5 Maker	0/1 테스트 99.5% 맞힘	22분 47초 ▲ 9% 빠름	5.39M ▼ 122% 많음	\$23.81 ▼ 236% 비쌘

하위 AI를 소형 모델로 바꾸면 돈은 어디로 가나 · 묶음 과제(3개)

메인은 Opus 5.5로 두고 하위 AI 모델만 바꿨다. 공식 단가가 Opus의 1/40인 GPT-6 Luna를 쓰면 총 요금이 \$6.81에서 \$3.37로 ▲ 50% 줄어듦, 대신 시간은 9분 40초에서 25분 4초로 2.6배 걸렸다. 하위 AI 묶은 \$0.15(총액의 4%)뿐이고, 나머지는 느린 하위 AI를 기다리며 검수한 메인 Opus 비용이다. 그래서 절감 폭이 단가 차이보다 작다.

API 환산 요금 — 주황 진한 쪽은 메인 Opus, 연한 쪽은 하위 AI (2회 평균)

메인 혼자		\$3.12
Opus 하위 AI		\$6.81
Luna 하위 AI		\$3.37

어려운 과제에서도 Luna 하위 AI는 \$2.90(하위 AI 몫 \$0.18)로 Opus 하위 AI \$4.47보다 싼지만, 1회뿐이고 숨김 테스트 195/196로 만점을 놓쳤다. 비용 분해는 세션 기록 토큰을 공식 단가로 다시 계산한 값이며 런타임 기록 요금과 일치한다.

무엇을 고쳤나 (문제 → 고친 것 → 효과)

1. 문서 카탈로그를 필요 시 조회로 이동

문제: 쓰지 않는 문서 목록을 매 요청마다 싣고 있었다.

첫 요청 크기(토큰) — 짧을수록 좋음

33,743 → 31,267 (-7%)

2. 위임 규칙을 발주 직전에만 로드

문제: 하위 AI를 부르지 않는 대부분의 작업에서도 위임 규칙 전체를 읽고 있었다.

첫 요청 크기(토큰) — 짧을수록 좋음

31,267 → 28,891 (-8%)

3. 독립 읽기 묶음 · 거절된 편집 재전송 규칙

문제: 파일을 하나씩 따로 읽고, 편집이 거절되면 다시 읽느라 요청 수가 늘었다.

보통 과제 AI 사용량 (GPT-6 Luna, 3회 가운데 값) — 짧을수록 좋음

181,000 → 137,000 (-24%)

4. 측정 조건 결함 수정

문제: 과제 폴더의 규칙 파일(약 3.5만 자)이 CUELO와 Codex에만 자동으로 읽혀 요청마다 약 2.8만 토큰이 더 들었다. 모든 도구에서 숨겨 조건을 맞췄다.

어려운 과제 CUELO 시간(초) — 짧을수록 좋음

530 → 395 (-25%)

5. 규칙 문서를 영어 사본으로

문제: 한국어 규칙은 토큰당 약 1.2자라 같은 내용이 영어보다 3~4배 비쌌다. 원문이 바뀌면 사본 대신 원문을 싣는다.

규칙 두 파일 크기(토큰) — 짧을수록 좋음

9,167 → 6,606 (-28%)

6. 브라우저·데스크톱 도구 설명을 필요 시 읽기

문제: 코드 작업에서는 거의 쓰지 않는 도구 설명 1.4만 자를 매 요청에 싣고 있었다.

첫 요청 크기(토큰, 실험 사본) — 짧을수록 좋음

48,676 → 43,050 (-12%)

7. 장기 프로세스 설명을 필요 시 읽기

문제: 서버·감시 프로세스를 띄울 때만 필요한 설명이었다.

첫 요청 크기(토큰, 실험 사본) — 짧을수록 좋음

43,050 → 42,280 (-2%)

8. 출력 한도 64K 제한 풀기

문제: 긴 과제에서 AI가 생각만 하다 응답 한도 64K에 걸려 코드 한 줄 못 쓰고 끊겼다(613초 손실). 공식 한도 128K를 쓰게 했다.

초대형 과제 전부 위임 CUELO 시간(초) — 짧을수록 좋음

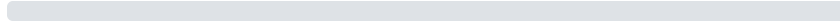


1,539 → 973 (-37%)

9. 끊긴 생각을 버리지 않고 이어 가기

문제: 한도에 걸려 끊기면 기록 요약이 그 생각을 지워, AI가 같은 고민을 처음부터 다시 했다(421초). 문맥이 작으면 요약 대신 짧은 안내를 붙여 이어 간다.

같은 과제에서 한도 중단 횟수 — 짧을수록 좋음



1 → 0 (-100%)

10. 첫 요청 고정부 재측정(Opus)

문제: 매 요청에 싣는 규칙·도구 설명이 Claude Code보다 컸다. 지난 축소 작업들의 누적 효과다.

첫 요청 크기(토큰, Opus 5.5) — 짧을수록 좋음



46,705 → 27,496 (-41%)

용어 풀이

AI 사용량

AI가 읽고 쓴 글자 양(토큰)을 요금 비중으로 환산한 값이다. 새로 읽은 양 + 0.1×이미 기억해 둔 부분 다시 읽기 + 1.25×기억해 두기 + 8×AI가 쓴 양. 비율은 가정값이고 모든 모델에 같게 적용한다. 같은 모델끼리는 적을수록 싸지만, 모델이 다르면 토큰 단가가 수십 배 달라 이 값으로 돈을 비교할 수 없다(아래 API 환산 요금을 본다). 위임한 경우 하위 AI가 쓴 양도 합친다.

맞힌 테스트 (예: 958/960)

AI가 만든 프로그램을 검사하는 자동 채점 문항 수다. 문항 하나는 "이 입력을 넣으면 이 답이 나와야 한다" 같은 확인 한 가지다. 예를 들어 초대형 과제에는 문항이 960개 있고, 958/960은 960개 중 958개를 통과했다는 뜻이다. 문항은 AI가 작업 중에 볼 수 없게 숨겨 두었다가 끝난 뒤에만 돌렸다. 시험지를 미리 보지 못하게 한 채점이다.

정답 횟수

같은 과제를 여러 번(기본 3회, 위임·대규모는 2회) 돌려 채점 테스트를 모두 통과한 횟수.

Maker·서브에이전트

메인 AI가 일을 나눠 맡기는 하위 AI.

high / medium

AI에게 생각을 얼마나 깊게 시키는지 설정. high가 더 깊고 느리다.

API 환산 요금

기록된 토큰을 각 회사 공식 API 단가(1M 토큰당 표준 요금)로 곱한 값이다. 측정은 월정액 구독으로 돌렸으므로 실제 청구액이 아니라 "API로 냈다면 얼마인가"다. 모델마다 따로 계산해 합친다. 단가 출처(2026-09-25 확인): [Anthropic Opus 5.5](#)(입력 \$4 · 출력 \$20 · 캐시 읽기 \$0.20), [OpenAI 가격표](#)(GPT-6 Luna 입력 \$0.10 · 출력 \$0.50, Daybreak Blue = gpt-5.6-sol 입력 \$4 · 출력 \$20), [DeepSeek 가격표](#)(V4.1 Flash 입력 \$0.15 · 출력 \$0.60, 혼잡 시간 2배). Claude Code는 도구가 직접 남긴 요금을 쓴다. Codex는 요청별 기록이 없어 긴 문맥 할증을 넣지 못했고 하위 AI 몫이 빠질 수 있어 실제보다 낮을 수 있다.

자세히 보기 (측정 방법)

순위는 정답률 → 시간 → 사용량 순이다. 돈 비교는 API 환산 요금 열을 본다. 그래프의 가는 선은 최소~최대 범위다. CUELO 시간은 세션 유휴 대기를 뺀 실제 작업 시간이다. 과제 폴더의 규칙 파일은 모든 도구에서 숨겨 같은 조건을 맞췄다. 총 76회 실행.